1. Name of Project

KinaseCondensateDB: A Relational Database for Human Kinase and Biomolecular Condensate Interaction Mapping

2. Names of Members of Group Working on Project

- Jinjie (Jay) Liu
- Jason Xu
- Yimeng (Donald) Dong

3. Name of Professor or Other Person Supervising This Work and Contact Information

Name: Berk Tuzcu
Organization: IRIS Kinetics
Role: Lead R&D Software Engineer
Relationship to Project: External industry supervisor providing dataset access and project guidance
Email: Berk@iriskinetics.com
Phone: 888-474-7008

4. Brief Description of Project

KinaseCondensateDB is a relational biological database designed to organize and explore the interactions between human kinase proteins and biomolecular condensates. The database integrates human kinase records sourced from UniProtKB/Swiss-Prot with condensate annotation data including condensate master records, protein-condensate association mappings, condensate-related biological molecules and modifications (c-mods), and condensatopathy evidence linking specific condensates to human diseases.

The goal of the project is to create a system that allows users to efficiently query kinase-condensate associations, explore the composition of individual condensates, and investigate disease-linked condensate dysregulation events supported by published literature. Users will be able to search by UniProt accession, gene name, condensate identifier, or disease name, and retrieve structured result sets drawn from an integrated relational schema spanning nine tables.

The database will also provide graphical summaries of query results, downloadable result files in CSV format, and direct links to UniProt and PubMed entries, creating a focused platform for exploring kinase biology in the context of biomolecular condensate function and disease.

5. Three Examples of Questions That Will Be Answered by the Database

1. Which biomolecular condensates are associated with a specific human kinase, and what are their condensate types and confidence scores?
2. Which human kinases are present within a specific biomolecular condensate, and what are their UniProt accessions, gene names, and sequence lengths?
3. Which biomolecular condensates are linked to a specific disease or dysregulation type, and what is the supporting literature evidence (PMID) for each association?

6. Description of User Accessibility

The database will be accessible through a private web interface intended for the project team, the course instructor, and the external supervisor for demonstration and evaluation purposes. Access to the interface will require login credentials and password protection. Authorized read-only users will be able to perform standard search, browsing, visualization, and download operations using UniProt accessions, gene names, condensate identifiers, or disease names. Users without approved credentials will not be able to access the system. Administrative functions such as updating records, modifying annotation tables, or importing new data will be restricted to the project developers.

7. Description of Data

KinaseCondensateDB integrates data from two primary sources. The first source is UniProtKB/Swiss-Prot, from which reviewed human kinase records are retrieved using the query organism_id:9606 AND reviewed:true AND keyword:KW-0418. This source provides protein accessions, entry names, gene names, organism information, sequence lengths, and amino acid sequences for 632 human-reviewed kinase entries.

The second source is a curated condensate annotation dataset provided to the project through the external supervisor at IRIS Kinetics as versioned project input files. This dataset comprises five files: a condensate master table (`condensates.csv`), a protein-to-condensate mapping file (`protein2cdcode_v2.1.tsv`), a c-mod annotation table (`c-mods.csv`), a condensatopathy evidence table (`condensatopathies.csv`), and a protein metadata table (`proteins.csv`). These files provide condensate identifiers, condensate types, species taxonomy identifiers, confidence scores, c-mod names, biomolecular types, phenotypic classes, disease names, dysregulation types, condensate markers, and PubMed identifiers for literature-supported associations.

All source datasets are maintained as versioned input files with stable identifiers used as relational anchors, including UniProt accessions, condensate UIDs, and PMIDs. During preprocessing, field names are standardized and multi-valued entries are normalized into relation-compatible structures before import into the relational schema.

Data objects stored in the database include:

- Protein
- Kinase
- Condensate
- Protein-Condensate association
- CMod (condensate-related biological molecule or modification)
- Condensate-CMod association
- Disease
- Condensate-Disease association
- Publication

8. Special Software Needed

No specialized commercial software is required for this project. If the project is deployed on bioed-new, the only requested software support beyond a standard Python environment would be MariaDB together with the Python packages needed for the web application and visualization layer, specifically Flask and Matplotlib. If these packages are already available on bioed-new, then no additional software installation will be needed.

9. Preliminary Tasks

1. Acquire the reviewed human kinase dataset from UniProtKB/Swiss-Prot and collect the curated condensate annotation input files provided through the project supervisor.
2. Standardize field names, clean input records, and normalize multi-valued entries so that proteins, condensates, c-mods, diseases, and publication evidence can be loaded into a relational schema.
3. Finalize a relational schema for Protein, Kinase, Condensate, Protein_Condensate, CMod, Condensate_CMod, Disease, Condensate_Disease, and Publication, including keys, foreign keys, and required indexes.
4. Implement the database tables, relationship constraints, and indexes in MariaDB.
5. Populate the database with cleaned kinase, condensate, association, c-mod, disease, and publication evidence records.
6. Implement representative SQL queries for kinase-centered, condensate-centered, and disease/evidence-centered retrieval.
7. Develop a Flask-based web interface for authenticated searching, browsing, and viewing query results.
8. Add outbound links to UniProt and PubMed for protein and evidence records shown in the interface.
9. Generate graphical summaries of selected query outputs using Python/Flask.
10. Implement downloadable result export in CSV and, if feasible, TSV format, and test the integrated system end-to-end.

10. Preliminary Database Structure

Entities and primary keys:

Protein: primary key **protein_id**; unique key **uniprot_accession**

Kinase: primary key **protein_id** (also foreign key to Protein)

Condensate: primary key **condensate_id**; unique key **condensate_uid**

Protein_Condensate: primary key **protein_condensate_id**; unique on (**protein_id**, **condensate_id**)

CMod: primary key **cmod_id**; unique key **cmod_name**

Condensate_CMod: primary key **condensate_cmod_id**

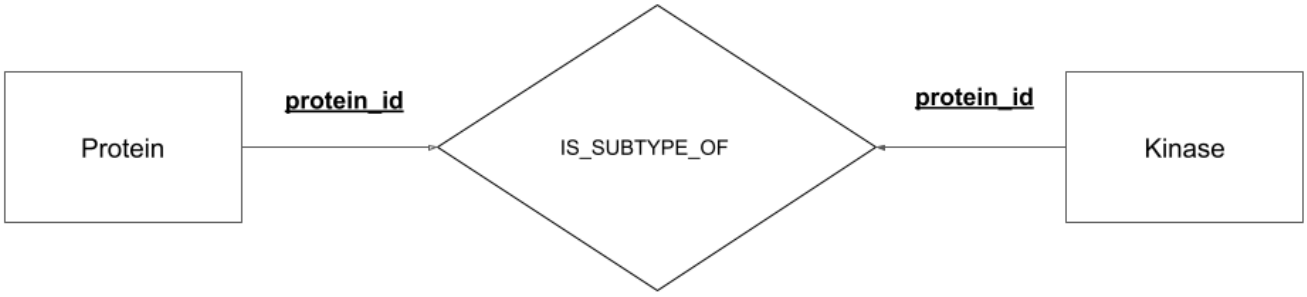
Disease: primary key **disease_id**; unique key **disease_name**

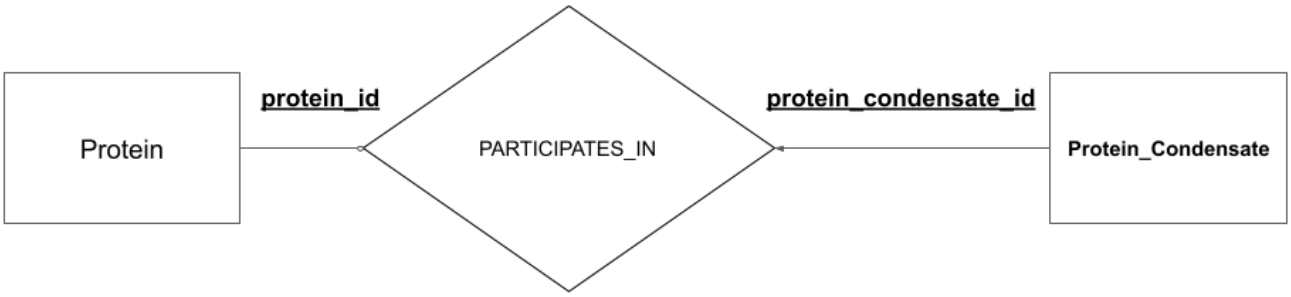
Condensate_Disease: primary key **condensate_disease_id**

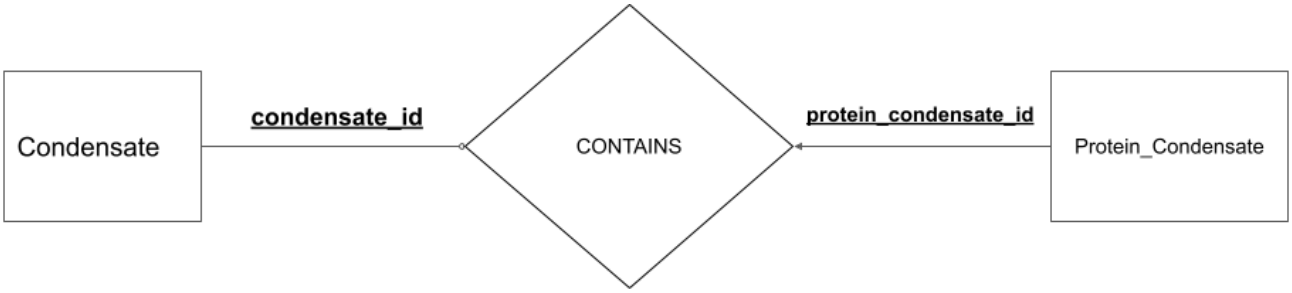
Publication: primary key **pmid**

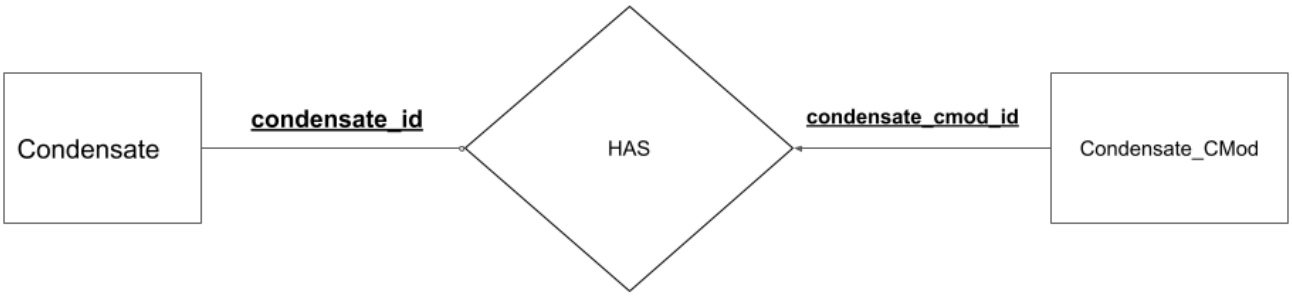
Protein	Kinase	Condensate	Protein_Condensate
---	---	---	---
protein_id (PK)	protein_id (PK/FK)	condensate_id (PK)	protein_condensate_id (PK)
uniprot_accession (UK)	entry_name	condensate_uid (UK)	protein_id (FK)
gene_name	organism_name	condensate_name	condensate_id (FK)
protein_name	sequence_length	condensate_type	evidence_source
species_name	sequence	species_tax_id	
biomolecular_condensate_count	reviewed_flag	confidence_score	
synthetic_condensate_count		has_dna	
		has_rna	

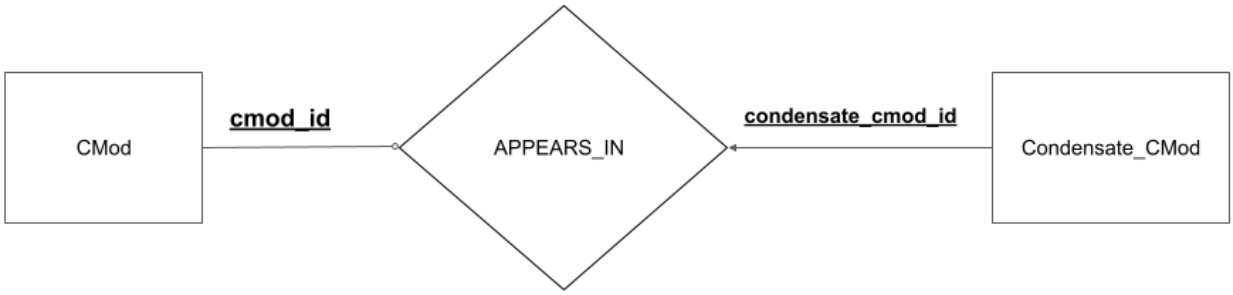
CMod	Condensate_CMod	Disease	Condensate_Disease	Publication
---	---	---	---	---
cmod_id (PK)	condensate_cmod_id (PK)	disease_id (PK)	condensate_disease_id (PK)	pmid (PK)
cmod_name (UK)	condensate_id (FK)	disease_name (UK)	condensate_id (FK)	
biomolecular_type	cmod_id (FK)		disease_id (FK)	
phenotypic_class	pmid (FK)		pmid (FK)	
			dysregulation_type	
			condensate_markers	

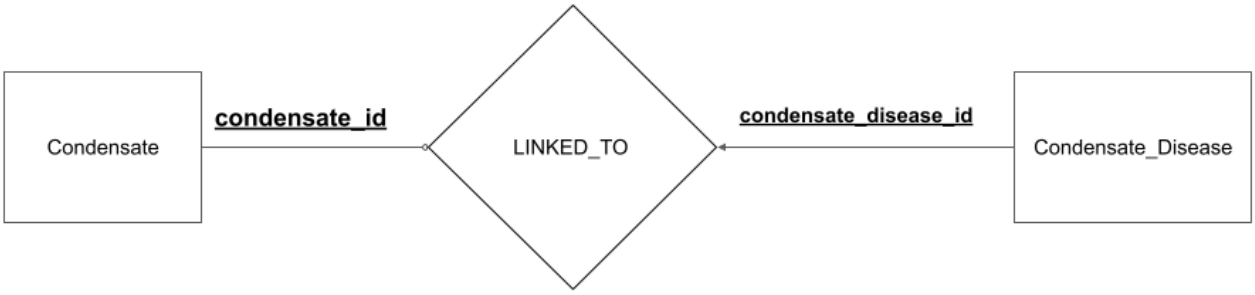


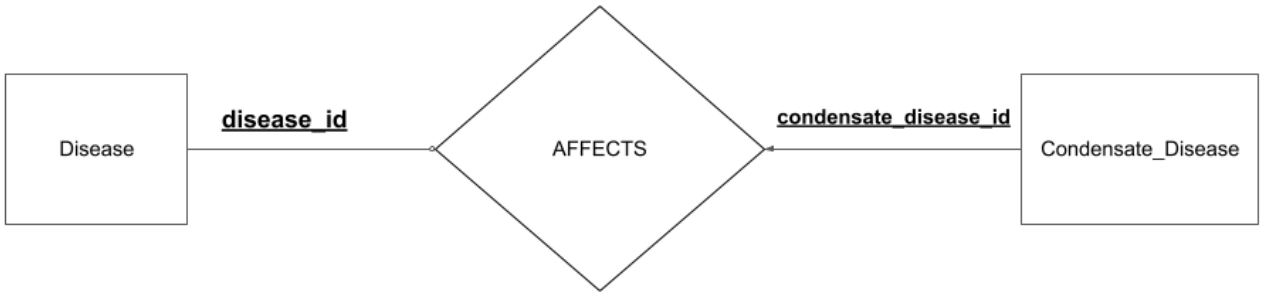


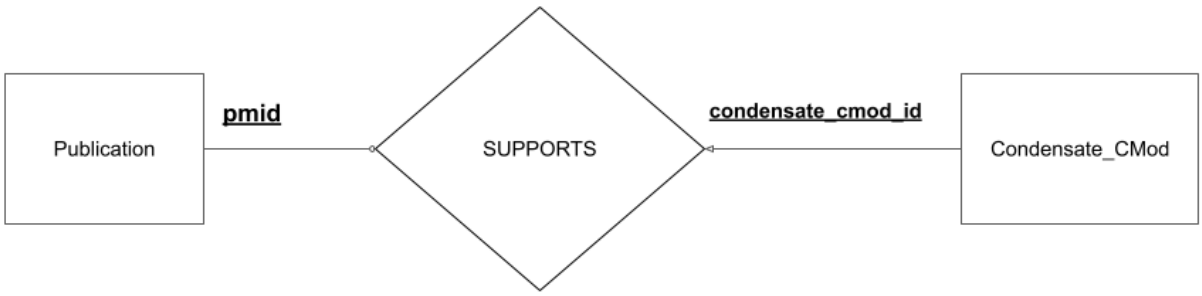


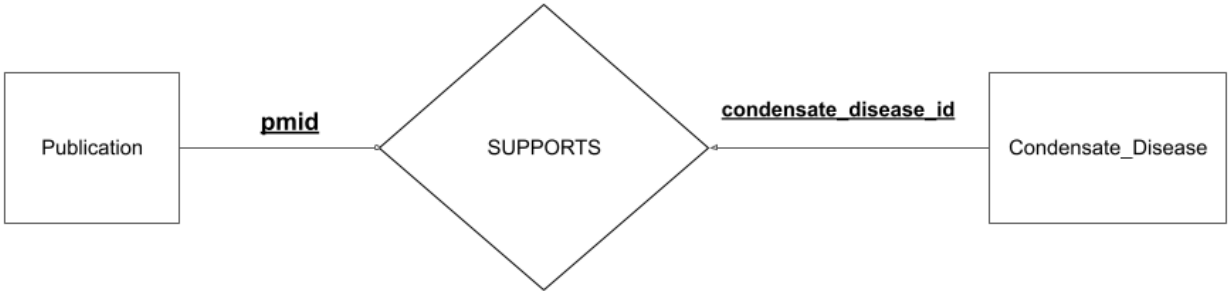



CMod
cmod_id
APPEARS_IN
condensate_cmod_id
Condensate_CMod


Condensate
condensate_id
LINKED_TO
condensate_disease_id
Condensate_Disease


Disease
disease_id
AFFECTS
condensate_disease_id
Condensate_Disease


Publication
pmid
SUPPORTS
condensate_cmod_id
Condensate_CMod



11. Table Descriptions Including Keys, Foreign Keys, and Indexed Fields

Table: Protein
Stores the base record for every protein in the database. This table serves as the central entity from which kinase-specific information and condensate associations are extended.

- Primary key: protein_id
- Unique key: uniprot_accession
- Foreign keys: none
- Indexed fields: PRIMARY KEY (protein_id); UNIQUE BTREE (uniprot_accession); BTREE (gene_name); BTREE (species_name)

Table: Kinase
Stores human-reviewed kinase-specific fields as a subtype of Protein. Every row in this table must correspond to an existing row in Protein.

- Primary key: protein_id
- Foreign keys: protein_id -> Protein.protein_id
- Indexed fields: PRIMARY KEY (protein_id); BTREE (entry_name); BTREE (reviewed_flag)

Table: Condensate
Stores the master record for each biomolecular condensate, including type classification, species, and confidence score.

- Primary key: condensate_id
- Unique key: condensate_uid
- Foreign keys: none
- Indexed fields: PRIMARY KEY (condensate_id); UNIQUE BTREE (condensate_uid); BTREE (condensate_name); BTREE (condensate_type); BTREE (species_tax_id)

Table: Protein_Condensate
Stores the many-to-many association between proteins and condensates. Each row records one protein-condensate pair.

- Primary key: protein_condensate_id
- Foreign keys: protein_id -> Protein.protein_id; condensate_id -> Condensate.condensate_id
- Indexed fields: PRIMARY KEY (protein_condensate_id); UNIQUE BTREE (protein_id, condensate_id); BTREE (condensate_id); BTREE (protein_id)

Table: CMod
Stores records for condensate-related biological molecules or modifications.

- Primary key: cmod_id
- Unique key: cmod_name
- Foreign keys: none
- Indexed fields: PRIMARY KEY (cmod_id); UNIQUE BTREE (cmod_name); BTREE (biomolecular_type); BTREE (phenotypic_class)

Table: Condensate_CMod
Stores the many-to-many association between condensates and c-mods, with optional literature support.

- Primary key: condensate_cmod_id
- Foreign keys: condensate_id -> Condensate.condensate_id; cmod_id -> CMod.cmod_id; pmid -> Publication.pmid
- Indexed fields: PRIMARY KEY (condensate_cmod_id); UNIQUE BTREE (condensate_id, cmod_id, pmid); BTREE (cmod_id); BTREE (pmid)

Table: Disease
Stores the master record for each condensatopathy or disease associated with condensate dysfunction.

- Primary key: disease_id
- Unique key: disease_name
- Foreign keys: none
- Indexed fields: PRIMARY KEY (disease_id); UNIQUE BTREE (disease_name)

Table: Condensate_Disease
Stores the many-to-many association between condensates and diseases, including dysregulation type, condensate markers, and supporting PMID.

- Primary key: condensate_disease_id
- Foreign keys: condensate_id -> Condensate.condensate_id; disease_id -> Disease.disease_id; pmid -> Publication.pmid
- Indexed fields: PRIMARY KEY (condensate_disease_id); BTREE (condensate_id, disease_id); BTREE (disease_id); BTREE (dysregulation_type); BTREE (pmid)

Table: Publication
Stores PubMed identifiers as evidence anchors for association records. All evidence-linked rows in Condensate_CMod and Condensate_Disease reference this table.

- Primary key: pmid

- Foreign keys: none
- Indexed fields: PRIMARY KEY (pmid)

12. Sample SQL Queries

Representative SQL queries have been designed to support the primary functions of KinaseCondensateDB. These queries illustrate how the relational schema will be used to retrieve kinase-centered, condensate-centered, and disease/evidence-centered result sets through the MariaDB backend.

Three Sample SQL Queries for Common Functions of the Database

Query 1: Retrieve biomolecular condensates associated with a queried human kinase

Function. Given a UniProt accession as input, this query returns the biomolecular condensates associated with the corresponding human kinase. This is the primary kinase-centered lookup function of the database.
Tables used: `Protein`, `Kinase`, `Protein_Condensate`, `Condensate`
Indexes used: `UNIQUE BTREE (Protein.uniprot_accession)` for input predicate; `BTREE (Protein_Condensate.protein_id)` for join; primary keys on `Kinase` and `Condensate`

```
SELECT
    p.uniprot_accession,
    p.gene_name,
    p.protein_name,
    c.condensate_uid,
    c.condensate_name,
    c.condensate_type,
    c.confidence_score
FROM Protein AS p
INNER JOIN Kinase AS k
    ON k.protein_id = p.protein_id
INNER JOIN Protein_Condensate AS pc
    ON pc.protein_id = p.protein_id
INNER JOIN Condensate AS c
    ON c.condensate_id = pc.condensate_id
WHERE p.uniprot_accession = :uniprot_accession
ORDER BY c.confidence_score DESC, c.condensate_name ASC;
```

Query 2: Retrieve human kinases associated with a queried biomolecular condensate

Function. Given a condensate UID as input, this query returns the human kinases associated with that condensate. This is the primary condensate-centered browsing function of the database.
Tables used: `Condensate`, `Protein_Condensate`, `Protein`, `Kinase`
Indexes used: `UNIQUE BTREE (Condensate.condensate_uid)` for input predicate; `BTREE (Protein_Condensate.condensate_id)` for join; primary keys on `Protein` and `Kinase`

```
SELECT
    c.condensate_uid,
    c.condensate_name,
```

```
    p.uniprot_accession,
    p.gene_name,
    p.protein_name,
    k.entry_name,
    k.sequence_length
FROM Condensate AS c
INNER JOIN Protein_Condensate AS pc
    ON pc.condensate_id = c.condensate_id
INNER JOIN Protein AS p
    ON p.protein_id = pc.protein_id
INNER JOIN Kinase AS k
    ON k.protein_id = p.protein_id
WHERE c.condensate_uid = :condensate_uid
ORDER BY p.gene_name ASC, p.uniprot_accession ASC;
```

Query 3: Retrieve disease-associated condensates and supporting evidence for a queried disease

Function. Given a disease name as input, this query returns the condensates linked to that disease, along with dysregulation type, condensate markers, and supporting PMIDs. This is the primary disease- and evidence-centered retrieval function of the database.
Tables used: `Disease`, `Condensate_Disease`, `Condensate`, `Publication`
Indexes used: `UNIQUE BTREE (Disease.disease_name)` for input predicate; `BTREE (Condensate_Disease.disease_id)` for join; `BTREE (Condensate_Disease.pmid)` for publication join

```
SELECT
    d.disease_name,
    c.condensate_uid,
    c.condensate_name,
    cd.dysregulation_type,
    cd.condensate_markers,
    pub.pmid
FROM Disease AS d
INNER JOIN Condensate_Disease AS cd
    ON cd.disease_id = d.disease_id
INNER JOIN Condensate AS c
    ON c.condensate_id = cd.condensate_id
LEFT JOIN Publication AS pub
    ON pub.pmid = cd.pmid
WHERE d.disease_name = :disease_name
ORDER BY c.condensate_name ASC, pub.pmid ASC;
```

13. External Data Processing

The database itself will perform relational storage, indexed retrieval, table joins, and SQL-based querying in MariaDB. External data processing will be limited to operations performed by the Flask-based web application layer rather than by the database engine itself.

User input normalization will be carried out in the web interface before SQL queries are submitted to MariaDB. This processing will include trimming user input, validating expected field formats, and mapping interface selections to database query parameters.

Result pagination and interface-level formatting will also be handled outside the database by the web application. These steps are intended to improve the usability of result pages without altering the underlying stored data.

Aggregation of SQL query output into summary tables for visualization will be performed in Python. These summarized result sets will then be used to generate graphical output files for display on selected results pages.

Graphic generation will be carried out in Flask/Python, and at least one graphical view will be produced as a file saved by the application and displayed to the user. This external processing supports the requirement that at least one database view use a graphics display based on a file produced and saved by the Flask program.

Export of query results into downloadable CSV and, optionally, TSV files will also be handled by the web interface outside the database engine. The exported files will contain the result set currently displayed to the user, formatted for downstream inspection or reuse.

Construction of outbound links to external resources such as UniProt entries and PubMed records will be performed by the application layer using stored identifiers such as UniProt accessions and PMIDs. This design keeps link generation separate from core relational storage while still supporting the requirement that the database provide links to other databases.

No specialized external analytical pipeline such as BLAST is currently planned; external processing is limited to interface-driven formatting, aggregation, visualization, and file export.

14. External Database Links

The database will provide direct outbound links to external biological databases and literature resources from its result pages. These links will allow users to move from internally stored records to authoritative source entries for protein annotation and literature evidence.

For protein and kinase records, the database will provide links to **UniProt** using the stored `uniprot_accession` field. These links will be displayed on protein-centered and kinase-centered result pages so that users can access curated protein names, sequence annotations, and related biological information.

For evidence-supported condensate or disease association records, the database will provide links to **PubMed** using the stored `pmid` field. These links will appear on evidence-oriented output pages, allowing users to inspect the original supporting literature for disease associations and condensate-related findings.

If additional annotation identifiers are incorporated during later implementation, the database may also provide links to **PDB**, **Pfam**, **InterPro**, **IntAct**, and **Complex Portal**. These external links will be generated only when corresponding identifiers are available in the stored annotation layer, so that each outbound link is directly supported by a field in the database.

In the current proposal design, the required external links are centered on **UniProt** for protein and kinase records and **PubMed** for literature-supported evidence records. This strategy is sufficient for the proposal

stage because it supports both biological entity lookup and evidence traceability while remaining consistent with the planned schema and query functions.

15. Graphical Output

The database will provide graphical output to help users interpret query results more efficiently than by reading tables alone. In addition to tabular result pages, selected query outputs will be summarized visually through figures generated by the Flask/Python application and displayed on the results page.

At least one graphical output will be created as a file produced and saved by the Flask program, in accordance with the project requirement for graphical data display. The graphical views will be derived from SQL query results returned by MariaDB and then converted into summary plots by the application layer before being presented to the user.

Example 1: Bar chart of biomolecular condensate counts for queried kinases. The x-axis will represent kinase identifiers such as gene names or UniProt accessions, and the y-axis will represent the number of biomolecular condensates associated with each kinase. This figure will allow users to compare the relative breadth of condensate associations among selected kinases. The input will be drawn from the result set of Sample SQL Query 1.

Example 2: Bar chart of kinase counts associated with a queried condensate. The queried condensate will serve as the reference entity, and the plot will display the number of kinases associated with that condensate. This figure will provide a compact visual summary of the kinase composition of a condensate-centered result set. The input will be drawn from the result set of Sample SQL Query 2.

Example 3: Disease-oriented summary plot of condensate associations. For disease- or dysregulation-based queries, the database will generate a figure showing the number of condensates linked to each disease or grouped by dysregulation type. This graphical output will help users identify which disease categories are most strongly represented in the stored evidence. The input will be drawn from the result set of Sample SQL Query 3.

All graphical outputs will be generated from processed query results, aggregated in Python, saved as PNG files by the application, and displayed on the relevant results page. These graphical views are intended to complement the underlying tabular outputs rather than replace them.

16. Data Download Function

The database will include a data download function that allows users to save query results generated through the web interface. Users will download processed result sets returned by the database in response to specific searches, rather than raw source files used during database construction.

The primary downloadable output will consist of tabular query results derived from the main database functions. These downloadable result sets will include kinase-to-condensate search results from Sample SQL Query 1, condensate-to-kinase search results from Sample SQL Query 2, and disease- and evidence-centered query results from Sample SQL Query 3.

For kinase-centered searches, downloadable files will include fields such as UniProt accession, gene name, protein name, condensate UID, condensate name, condensate type, and confidence score. For condensate-centered searches, downloadable files will include condensate UID, condensate name, UniProt accession, gene name, protein name, kinase entry name, and sequence length. For disease and evidence searches,

downloadable files will include disease name, condensate UID, condensate name, dysregulation type, condensate markers, and PMID.

The primary download format will be **CSV**, since it is widely supported for tabular biological data and can be opened directly in spreadsheet software or imported into downstream analysis tools. An optional **TSV** format may also be provided. In addition, selected graphical output files generated by the application may be made available for download in **PNG** format.

The download function will be implemented through the web interface and will export the currently displayed result set rather than the entire database. The downloaded file will therefore correspond directly to the query the user has performed and reviewed on screen. This design satisfies the final proposal requirement for a data download function and is aligned with the course expectation that the system support data download into one or more file formats.